



Application de gestion de bibliothèque personnelle

Réalisé par : **Douae Bousnane**

Formation : **1PRO2**

École : **SUPINFO Lille**

Année : **2025 - 2026**

Date de rendu : **14 juin 2026**

Sommaire

1. Schéma relationnel
2. Structure du projet
3. Installation backend
4. Installation frontend
5. Variables d'environnement
6. Choix techniques
7. Guide utilisateur

1. Schéma relationnel

Trois ressources principales, toutes rattachées à l'utilisateur qui les a créées. Chaque livre peut avoir un auteur et appartenir à plusieurs collections.

```
User
|-- Livre      (1-N)
|   |-- Auteur  (oneToOne, optionnelle)
|   |-- Collection (manyToMany)
|-- Auteur      (1-N)
+-- Collection (1-N)
```

Livre

- **titre** — string, obligatoire
- **description** — text
- **couverture** — string (URL)
- **statut** — enum : a_lire / en_cours / termine
- **note** — integer (1 à 5)
- **avis** — text
- **auteur** — relation oneToOne avec Auteur
- **collections** — relation manyToMany avec Collection
- **users_permissions_user** — relation oneToOne avec User

Auteur

- **nom** — string, obligatoire
- **prenom** — string
- **users_permissions_user** — relation oneToOne avec User

Collection

- **nom** — string, obligatoire
- **users_permissions_user** — relation oneToOne avec User

Le rattachement utilisateur sert au filtrage côté contrôleur : chaque *find* n'expose que les ressources dont *users_permissions_user* correspond à l'utilisateur connecté.

2. Structure du projet

```
supbook-backend/  
|-- config/  
|-- database/  
|-- public/  
|-- src/  
|   |-- api/  
|   |   |-- auteur/      (content-types, controllers, routes, services)  
|   |   |-- collection/  (content-types, controllers, routes, services)  
|   |   +-- livre/      (content-types, controllers, routes, services)  
|   +-- index.js  
|-- types/  
|-- .env.example  
|-- package.json  
+-- README.md
```

```
supbook-frontend/  
|-- public/  
|-- src/  
|   |-- pages/  
|   |   |-- Bibliotheque.js  
|   |   |-- Auteurs.js  
|   |   |-- Collections.js  
|   |   |-- Connexion.js  
|   |   +-- Inscription.js  
|   |-- services/  
|   |   +-- api.js  
|   |-- App.js  
|   |-- index.js  
|   +-- index.css  
+-- package.json
```

3. Installation backend

Prérequis

- Node.js v20+
- npm v9+

Étapes

```
# 1. Se placer à la racine du backend
cd supbook-backend

# 2. Installer les dépendances
npm install

# 3. Copier le fichier d'exemple en .env
cp .env.example .env

# 4. Lancer Strapi en mode développement
npm run develop
```

Strapi est ensuite accessible sur <http://localhost:1337/admin>. Au premier lancement, il faut créer le compte super-admin via l'interface.

Permissions à activer

Dans **Settings** → **Users & Permissions** → **Roles** → **Authenticated**, activer pour les trois content-types **Livre**, **Auteur** et **Collection** les actions :

- find
- findOne
- create
- update
- delete

4. Installation frontend

Prérequis

- Node.js v20+
- npm v9+

Étapes

```
# 1. Se placer dans le dossier frontend  
cd supbook-frontend
```

```
# 2. Installer les dépendances  
npm install
```

```
# 3. Lancer l'application  
npm start
```

L'application est ensuite accessible sur <http://localhost:3000>.

Le backend Strapi doit tourner avant le frontend, sinon les appels API échouent.

5. Variables d'environnement

Copier *.env.example* en *.env* à la racine du backend. Au premier *npm run develop*, Strapi génère les secrets manquants.

```
HOST=0.0.0.0
PORT=1337
APP_KEYS=...
API_TOKEN_SALT=...
ADMIN_JWT_SECRET=...
TRANSFER_TOKEN_SALT=...
JWT_SECRET=...
DATABASE_CLIENT=sqlite
DATABASE_FILENAME=.tmp/data.db
```

Le fichier *.env* ne doit pas être commité — il est déjà dans *.gitignore*.

6. Choix techniques

J'ai pris **Strapi v5** parce que la consigne impose un CMS headless et que les routes REST sont générées automatiquement à partir des content-types. Ça m'a permis de me concentrer sur la logique métier qui ne sort pas de Strapi par défaut : le rattachement utilisateur au moment de la création, le filtrage par utilisateur dans le *find*, et le nettoyage des relations à la suppression d'un auteur (les livres concernés voient leur champ *auteur* remis à null pour éviter une référence morte).

Côté front, **React** parce que je le connais le mieux. Je n'ai pas utilisé *react-router-dom* : avec cinq pages au total, un *useState* dans *App.js* qui stocke la page active suffit, et ça évite une dépendance pour pas grand-chose.

Pas non plus d'*axios* ni de librairie HTTP : *fetch* est dispo dans le navigateur. Tous les appels passent par un helper *request()* dans *services/api.js* qui centralise l'ajout du token, la sérialisation JSON et le mapping des erreurs réseau vers un objet `{ erreur: ... }` que l'UI peut afficher. Ça évite de répéter le même *try / catch* dans chaque fonction.

Le **JWT** renvoyé par */auth/local* est stocké dans le *localStorage*, ce qui permet de rester connecté entre deux rafraîchissements. À la déconnexion, je le retire.

SQLite pour la base : c'est un rendu local, ça évite d'installer un Postgres juste pour ça, et la migration vers Postgres se ferait via la variable *DATABASE_CLIENT* sans toucher au code.

Pour le **style**, j'ai gardé du CSS inline directement dans les composants : la taille du projet ne justifiait pas l'ajout d'une lib de styling, et ça reste lisible. La police *Playfair Display* est chargée via Google Fonts dans *index.css*.

7. Guide utilisateur

Créer un compte

- Aller sur <http://localhost:3000>
- Cliquer sur **S’inscrire**
- Remplir le nom d'utilisateur, l'email et le mot de passe
- Redirection automatique vers la page de connexion

Se connecter

- Entrer l'email et le mot de passe
- Cliquer sur **Se connecter**
- Redirection automatique vers la bibliothèque

Ajouter un livre

- Cliquer sur + **Ajouter un livre**
- Le titre est obligatoire, le reste est optionnel
- Choisir un statut : À lire / En cours / Terminé
- Optionnel : ajouter une note (1-5) et un avis personnel
- Optionnel : associer un auteur et cocher une ou plusieurs collections
- Cliquer sur **Ajouter le livre**

Modifier ou supprimer un livre

- **Modifier** : ouvre une modale avec les mêmes champs que la création (auteur, statut, note, avis, collections multiples)
- **Supprimer** : confirmation puis suppression effective

Gérer les auteurs et collections

- Boutons **Auteurs** et **Collections** dans la barre de navigation
- Sur chaque page, un bouton d'ajout ouvre un formulaire
- Les auteurs et collections créés ici sont ensuite proposés dans le formulaire d'ajout / édition de livre

Recherche, filtres et tri

- **Recherche** : taper dans la barre pour filtrer les titres en temps réel
- **Statut** : Tous / À lire / En cours / Terminé
- **Auteur** et **Collection** : filtre par sélection dans la sidebar
- **Tri** : Par défaut, Titre A→Z, Meilleure note, Statut

Se déconnecter

- Cliquer sur **Se déconnecter** dans la nav
- Le token JWT est retiré du localStorage, retour à l'écran de connexion